

3-Year High-Paying Career Roadmap

Modern C++ + Qt/QML + Systems + Medical Imaging + Computer Vision + AI

Career Objective

Current: ■6.25 LPA **Target:** ■20+ LPA within approximately 3 years.

Build on existing professional C++/Qt experience rather than restart in a generic stack. Modern C++ remains the foundation, while Qt/QML, systems engineering, medical imaging, computer vision, and AI are added progressively.

Target Career Profile

Layer	Target capability
Core	Modern C++, STL, memory management, OOP, RAII, smart pointers
Application	Qt 6, Qt Widgets, QML, C++/QML integration
Systems	CMake, Linux, multithreading, debugging, profiling, performance
Design	SOLID, Design Patterns, architecture, system design
Domain	Medical Imaging, DICOM, DCMTK, ITK, VTK
Vision	OpenCV, image processing, computer vision
AI	Python, PyTorch, MONAI, deep learning
Deployment	ONNX, ONNX Runtime, TensorRT, production inference

YEAR 1 — Strong C++/Qt Engineer

Objective: Become highly interview-ready for C++/Qt and medical-imaging software roles.

Target: approximately ■9–12 LPA, depending on role and market.

Months 1–3 — Modern C++ Foundation

- OOP, inheritance, polymorphism, virtual functions, abstract classes, interfaces, constructors/destructors, const correctness, references and pointers.
- Stack vs heap, object lifetime, memory leaks, dangling pointers, ownership, RAII, `unique_ptr`, `shared_ptr` and `weak_ptr`.
- Copy/move constructor and assignment, Rule of 0/3/5, lvalue/rvalue and `std::move`.

Months 2–5 — STL + Modern C++

- vector, deque, list, map, unordered_map, set, unordered_set, stack, queue, priority_queue, iterators and algorithms.
- Lambdas, auto, range-based for, nullptr, enum class, constexpr, array, optional, variant and string_view.
- Understand C++11/14/17 and selected C++20 features well enough to explain practical usage.

Months 2–6 — Design Patterns (Start Now)

- Factory/Factory Method, Singleton, Observer, Strategy, Adapter, Builder, Command and State.
- For every pattern learn the problem, motivation, structure, C++ implementation, real-world use, Qt example and when not to use it.
- Qt connections: Observer-like Signals & Slots, Model/View, Factory-style creation, Strategy-style algorithms and State-based UI behavior.
- Do not claim professional use of a pattern unless it was actually used; be able to discuss it as a design approach.

Months 3–6 — DSA + Coding Interviews

- Arrays, strings, hashing, linked lists, stacks, queues, trees, BST, heaps and graphs.

- Sorting, searching, two pointers, sliding window, recursion, backtracking, BFS, DFS, greedy and basic dynamic programming.
- Aim for roughly 100–150 carefully selected interview problems rather than random high-volume practice.

Months 4–8 — Qt 6 + QML

- QObject, parent-child ownership, signals/slots, event loop, events, event filters, properties, Meta-Object System, MOC and Qt threading.
- Qt Widgets: Model/View, custom widgets, QPainter, Graphics View, delegates and layouts.
- QML: properties, bindings, signals, models, delegates, ListView, GridView, custom components, C++/QML integration and performance.
- Shift long-term positioning toward Qt 6 + C++ + QML + CMake.

Months 6–10 — Multithreading & Concurrency

- std::thread, mutex, lock_guard, unique_lock, condition_variable, future, promise, async, atomic and memory ordering.
- Race conditions, deadlocks, producer/consumer, thread pools and task queues.
- Apply concurrency to image loading, image processing, data recording and UI responsiveness.

Months 6–12 — CMake + Linux + Debugging

- CMake targets, linking, include directories, PUBLIC/PRIVATE/INTERFACE, find_package, FetchContent, install/export and toolchains.
- Linux processes, threads, shared/static libraries, dynamic linking, signals, permissions, shell and environment variables.
- GDB, core dumps, AddressSanitizer, UndefinedBehaviorSanitizer, Valgrind, stack traces and logging.
- Performance basics: CPU hotspots, memory usage, allocations, unnecessary copies, cache behavior and concurrency bottlenecks.

Months 8–12 — Software Design & System Design

- SOLID, dependency inversion, interfaces, modular architecture, layered architecture, API design, dependency management and maintainability.
- Start with smaller systems such as image-processing pipelines, logging systems, multithreaded data processors and image-viewer architecture.

Year 1 AI Track

Keep AI at approximately 10% of study time initially; do not let it weaken core C++/Qt preparation.

- Python fundamentals, NumPy, Matplotlib and OpenCV.
- Machine-learning concepts: supervised/unsupervised learning, train/validation/test, overfitting, classification, regression and evaluation metrics.

YEAR 1 PORTFOLIO PROJECT

Modern Medical Image Viewer

- Technology: C++17/20, Qt 6, QML, CMake, DICOM, DCMTK, OpenCV, VTK and multithreading.
- Features: DICOM loading, navigation, zoom, pan, window/level, measurements, annotations, image processing, threaded loading, C++/QML integration, logging and settings.
- Use public data and independently written code; never use proprietary company source code or data.

YEAR 2 — Senior-Level C++ + Medical Imaging + Computer Vision

Objective: Move from feature implementation toward subsystem ownership, architecture and specialization.

Target: approximately ■13–17 LPA, depending on role and market.

Months 13–18 — Advanced C++

- Templates, type traits, perfect forwarding, exception safety, undefined behavior, object lifetime, alignment, padding, endianness, binary data, serialization and ABI basics.
- Use legacy C++ experience to understand portability, memory representation and safe data handling.

Months 13–18 — Advanced Qt

- Qt architecture, event-loop behavior, QThread architecture, QtConcurrent, Model/View, QML performance, C++/QML architecture, networking and Qt testing.

Months 14–20 — Medical Imaging Deep Dive

- DICOM datasets, tags, VR, transfer syntax, SOP Class/Instance, Patient/Study/Series, UIDs, C-FIND, C-MOVE, C-STORE and PACS.
- Pixels, voxels, coordinate systems, window/level, interpolation, resampling, filtering, segmentation and registration.
- Deepen DCMTK, ITK, VTK and OpenCV based on real project needs.

Months 18–24 — Computer Vision

- Image filtering, edge detection, morphology, feature extraction, segmentation, object detection, classification, registration and image enhancement.

Months 18–24 — Deep Learning

- Neural networks, backpropagation, optimization, loss functions, CNNs, ResNet, U-Net, transfer learning, segmentation and detection.
- Use PyTorch and then explore MONAI for medical-imaging AI.

YEAR 2 PROJECT — Medical AI Imaging System

Pipeline: DICOM → preprocessing → AI model → segmentation/classification → result → C++/Qt/QML → visualization.

- Use public datasets only. The project demonstrates engineering integration rather than clinical diagnosis.
- Document architecture, model choice, performance, limitations and reproducibility.

YEAR 3 — ■20+ LPA Target

Objective: Become marketable as a Senior C++ Engineer, Senior C++/Qt Engineer, Medical Imaging Engineer, C++ Computer Vision Engineer or C++ AI Engineer.

Months 24–30 — Production AI

- ONNX, ONNX Runtime, TensorRT, model inference, CPU/GPU inference, optimization, quantization, latency, memory optimization and batching.
- Production pipeline: PyTorch → ONNX → C++ → ONNX Runtime → Qt/QML.
- Focus on integrating AI into native software rather than only training models.

Months 24–30 — Advanced System Design

- Design systems such as a medical image-processing platform with DICOM input, parser, validation, image cache, processing pipeline, worker pool, AI inference, result manager and Qt/QML visualization.
- Explain components, interfaces, threading, memory, error handling, performance, testing, logging and maintainability.

Months 24–36 — Interview Preparation

Area	Focus
C++	30% — Modern C++, STL, memory, templates, concurrency, lifetime, performance
DSA	15% — Arrays, strings, trees, graphs, hashing, recursion, DP
Qt/QML	15% — QObject, signals/slots, event loop, QThread, Model/View, QML, C++/QML
Design Patterns	10% — Factory, Strategy, Observer, Adapter, Builder, Command, State, Singleton
System Design	10% — Architecture, SOLID, modularity, concurrency, APIs
Medical Imaging	10% — DICOM, DCMTK, ITK, VTK, imaging concepts
AI/CV	10% — PyTorch, CNNs, segmentation, computer vision, ONNX, inference

Job-Switch Strategy

- 6–12 months: selectively interview for C++ Developer, C++ Software Engineer, Qt Developer, C++/Qt Engineer and Medical Imaging Software Engineer roles.
- 12–24 months: target roles requiring roughly 4–6 years of C++ experience, emphasizing Modern C++, Qt, multithreading, CMake, Linux and architecture.
- 24–36 months: target Senior C++ Software Engineer, Senior C++/Qt Engineer, Medical Imaging Engineer, Computer Vision Engineer, C++ Systems Engineer and C++ AI roles.
- Do not wait three years before interviewing. Use interviews to measure market readiness and identify gaps.

Weekly Study Schedule

Day	Focus
Monday	Modern C++ — 2 hours
Tuesday	Qt/QML — 2 hours
Wednesday	DSA + C++ coding — 2 hours
Thursday	Design Patterns + System Design — 2 hours
Friday	Multithreading + CMake/Linux — 2 hours
Saturday	Portfolio project — 3–4 hours
Sunday	AI + revision + interview preparation — 2 hours

Three Major Portfolio Projects

Project	Stack	What it proves
Modern Medical Image Viewer	C++, Qt 6, QML, CMake, DICOM, DCMTK, OpenCV, VTK	C++/Qt, imaging, UI, processing, visualization and concurrency
Medical AI Imaging System	Python, PyTorch, MONAI, OpenCV, DICOM	Medical AI, computer vision, segmentation/classification
C++ AI Inference System	C++20, CMake, ONNX Runtime, Qt/QML	Production AI integration and native inference

Final Career Positioning

Today: C++ Software Engineer — ■6.25 LPA — C++/Qt/QML — Automotive + Medical Imaging.

~1 year: Strong C++/Qt Engineer — Modern C++, Qt 6, CMake, multithreading, Design Patterns, Linux.

~2 years: Senior-level C++ Engineer + Medical Imaging + Computer Vision + Deep Learning.

~3 years: Senior C++ / AI Engineer — C++/Qt + Medical Imaging + Computer Vision + AI + Production Inference.

Core principle: Do not abandon C++ to chase AI. Build a differentiated profile by combining durable C++/systems engineering with Qt, medical imaging, computer vision and AI. No technology stack is permanently future-proof; keep strengthening fundamentals while adapting to AI advances.